



Факультет экономических наук

Базовая кафедра инфраструктуры
финансовых рынков

Москва

Библиотека **BeautifulSoup**

Разбираем **HTML**-код страницы и извлекаем из него нужные данные

Бакланова Валерия Сергеевна

преподаватель БКИФР, младший научный сотрудник ЦФИАНД



BeautifulSoup (bs4) — библиотека **Python** для парсинга **HTML**- и **XML**-документов. Она умеет находить нужные теги внутри страницы, перемещаться между ними, а также удалять и изменять содержимое.

Обычно **bs4** используют вместе с **Requests**: **Requests** загружает **HTML**-код страницы, а **BeautifulSoup** разбирает его на теги и позволяет удобно искать нужные данные.

```
pip install beautifulsoup4

import requests
from bs4 import BeautifulSoup

response = requests.get('https://адрес_веб-страницы_для_парсинга')
response.encoding = 'utf-8' # если текст читается некорректно
soup = BeautifulSoup(response.text, features='lxml')
```

Совет: если сервер часто блокирует запросы, сохраните **HTML**-страницу на диск и разбирайте её локально — это снижает число обращений к сайту и риск бана по **IP**.



`features` в конструкторе `BeautifulSoup()` задаёт парсер — движок, который разбирает **HTML**-код на дерево тегов:

Парсер	Преимущества	Недостатки
<code>html.parser</code>	встроен в Python , ставить не нужно	медленнее lxml , менее терпим к битому HTML
<code>lxml</code>	очень быстрый, терпим к битому HTML	нужно ставить отдельно: pip install lxml
<code>xml</code>	быстрый; единственный парсер XML	внешняя зависимость (тот же lxml)
<code>html5lib</code>	разбирает страницу так же, как браузер	очень медленный, отдельная зависимость

На практике: для больших объёмов парсинга обычно выбирают **lxml** — он заметно ускоряет разбор сотен и тысяч страниц.



Tag — тип объекта, которым **BeautifulSoup** представляет каждый **HTML**- или **XML**-тег исходного документа. У объекта есть атрибуты и методы для доступа к данным:

- `.name` — имя тега;
- `.attrs` — словарь атрибутов тега;
- `.find()` / `.find_all()` и другие — методы поиска тегов внутри или вокруг данного.

```
from bs4 import BeautifulSoup

html = "<div class='myclass'>Hello, world!</div>"
soup = BeautifulSoup(html, 'html.parser')
tag = soup.div

print(tag.name)    # div
print(tag.attrs)   # {'class': ['myclass']}
print(tag.string)  # Hello, world!
```

Важно: объекты **Tag** изменяемы — их имя, атрибуты и содержимое можно менять программно.



`.text` — свойство, которое возвращает весь текст внутри тега вместе с текстом всех его дочерних элементов, в виде одной непрерывной строки.

`.text` проще в использовании, но менее гибкое, чем `.get_text()` — оно не даёт настроить, как именно объединяются фрагменты текста.

```
html_string = """
<div>
    <p>Первый абзац.</p>
    <p>Второй абзац <span>со вложенным</span> текстом.</p>
</div>"""

soup = BeautifulSoup(html_string, 'html.parser')
print(soup.find('div').text)
```

Первый абзац.

Второй абзац со вложенным текстом.



Извлечение текста: `.get_text()`

6

`.get_text()` — метод, который тоже возвращает весь текст тега, но с параметрами для настройки:

```
.get_text(separator=" ", strip=False)
```

Параметр	Назначение
<code>separator</code>	строка-разделитель между текстом из разных тегов
<code>strip</code>	True — убирает начальные и конечные пробелы в каждом фрагменте

```
soup.find('div').get_text(separator=" | ")  
>>> Первый абзац. | Второй абзац | со вложенным | текстом.
```



	.text	.get_text()
Тип	свойство	метод
Настройка	нет параметров	separator, strip и другие
Когда использовать	нужен весь текст как есть	нужен читаемый, «причёсанный» текст

```
div_text = soup.find('div').get_text(separator=" | ")
print(div_text)
```

```
>>> | Первый абзац. |
>>> | Второй абзац | со вложенным | текстом. |
```

Правило: если нужен просто весь текст — берите **.text**; если нужен контроль над разделителями и пробелами — **.get_text()**.



`soup.find()` ищет **первый** элемент, удовлетворяющий заданным критериям. Если ничего не найдено — возвращает `None`.

```
soup.find(name, attrs={}, recursive=True, string, **kwargs)
```

Параметр	Назначение
<code>name</code>	имя тега для поиска: строка или регулярное выражение
<code>attrs</code>	словарь атрибутов искомого элемента
<code>recursive</code>	False — искать только среди прямых потомков; по умолчанию True — по всем вложенным
<code>string</code>	искать по текстовому содержимому тега
<code>**kwargs</code>	любой неизвестный именованный аргумент bs4 воспринимает как атрибут тега



.find(): поиск по id, классу и атрибутам

9

```
# Поиск по id  
soup.find('tag', id='your_id')
```

```
# Поиск по классу (class_ — с подчёркиванием, class зарезервировано в Python)  
soup.find('tag', class_='your_class')
```

```
# Поиск по пользовательскому атрибуту  
soup.find('tag', {'data-attribute': 'value'})
```

```
# Поиск сразу по двум пользовательским атрибутам  
soup.find('tag', {'data-attribute1': 'value1', 'data-attribute2': 'value2'})
```



.find(): более сложные условия

10

```
# Поиск по двум классам сразу
soup.find('tag', class_='class1 class2')
```

```
# Поиск по «динамическому» классу — известна лишь часть его названия, или класс формируется динамически
import re
soup.find('tag', class_=re.compile('your_dynamic_part'))
```

```
# Поиск по динамическому пользовательскому атрибуту
soup.find('tag', {'data-attribute': re.compile('part')})
```

```
# Поиск тега вообще без учёта атрибутов. Найдёт первый тег с указанным именем.
soup.find('tag')
```



`soup.find_all()` находит **все** подходящие элементы и всегда возвращает список — даже если найден один элемент или не найдено ни одного.

```
soup.find_all(name, attrs={}, recursive=True, string, limit, **kwargs)
```

Параметры такие же, как у `.find()`, плюс `limit` — ограничивает число найденных элементов.

```
# Найдёт все элементы хотя бы с одним из классов
soup.find_all('tag', class_=['class1', 'class2'])
```

Частый приём: сначала `.find()` находит родительский блок, а затем `.find_all()` ищет внутри него нужные элементы.



Результат `.find()` / `.find_all()` — это тоже объект(ы) **Tag**, внутри которых можно продолжать поиск:

```
# <p class="story">
#   <a class="hero" href="http://example.com/ilya" id="link1">Илья</a>
#   <a class="hero" href="http://example.com/alesha" id="link2">Алёша</a>
# </p>
```

```
first_story = soup.find('p', class_='story')
link2 = first_story.find(id='link2') # ищем уже внутри найденного тега
```

```
# Текст тега
print(link2.text)
>>> Алёша
```

```
# Атрибут тега — как ключ словаря
print(link2['href'])
>>> http://example.com/alesha
```

`tag.get("attribute_name", default=None)` — более безопасный способ получить значение атрибута: не вызывает ошибку, а возвращает **None**, если атрибута нет.

```
# <a href="https://example.com">Visit example.com</a>
a_tag = soup.find("a")
href_value = a_tag.get("href")
```



- Ищите теги в несколько шагов — от более крупных элементов к более мелким.
- Не полагайтесь на относительную вложенность элементов и их порядок на странице — опирайтесь на теги и их атрибуты.
- При поиске по атрибутам в первую очередь используйте `id` — он должен быть уникальным на странице (но бывают исключения).
- Если значение атрибута состоит из нескольких слов, выбирайте то из них, которое вряд ли изменится.
- Не используйте классы, связанные со стилями или обозначающие что-то непонятное (например, `class="col-md-3 mb-3 mt-1"`) — они ненадёжны для поиска.

```
# <a href="http://example.com/ilya" class="hero superhero" id="link1">Илья</a>
# <a href="http://example.com/alesha" class="superhero hero" id="link2">Алёша</a>

super_hero = soup.find_all(class_='superhero') # найдёт оба тега <a>
```



`soup.select()` находит все элементы, подходящие под **CSS-селектор** — тот же синтаксис, что используется в **CSS**-стилях и в консоли браузера.

	<code>.find()</code> / <code>.find_all()</code>	<code>.select()</code>
Результат	один элемент / список	всегда список
Гибкость	параметры name , attrs , string	любой CSS -селектор целиком

```
soup.select(selector, namespaces=None, limit)
```

Когда выбрать `.select()`: если условие поиска проще выразить одной **CSS**-строкой, чем набором параметров `find()`.



Что ищем	Селектор
Тег	<code>select("p")</code>
Класс	<code>select(".class")</code>
Идентификатор	<code>select("#id")</code>
Атрибут	<code>select("[attribute=value]")</code>
Тег с классом	<code>select("p.class")</code>
Вложенные элементы	<code>select("div span")</code>
Непосредственные потомки	<code>select("div > p")</code>
Несколько классов сразу	<code>select(".class1.class2")</code>
Псевдоклассы CSS	<code>select("p:first-of-type")</code>
Соседний элемент	<code>select("h1 + p")</code>

```
# Элементы, содержащие определённый текст
elements = soup.select("p")
elements_with_text = [e for e in elements if "нужный текст" in e.text]
```



Метод `.select_one()`

16

`soup.select_one()` выбирает по **CSS**-селектору только **первый** подходящий элемент — или `None`, если ничего не найдено. Полезен, когда заранее известно, что нужный элемент — единственный.

`.select()`

возвращает список всех найденных тегов

`.select_one()`

возвращает только первый найденный тег

```
select_one(selector)
```

Что ищем	Селектор
Тег с id	<code>select_one("#id")</code>
Непосредственный потомок	<code>select_one("div > p")</code>



requests-cache загружает **HTTP**-ответ от сервера, кеширует его и передаёт на обработку другим библиотекам. Полезна, когда парсер многократно обращается к одним и тем же страницам во время разработки и отладки — повторные запросы идут из кеша, а не с сайта.

```
pip install requests-cache

import requests_cache

# Запускаем сессию, кеширующую результаты загрузки страницы
session = requests_cache.CachedSession()
response = session.get('https://адрес_веб-страницы_для_парсинга')
```

```
# Очистить кеш
session.cache.clear()
```

```
# Список закешированных страниц
print(session.cache.urls)
```



tqdm позволяет реализовать прогресс-бар — удобно видеть, сколько страниц парсер уже обработал и сколько осталось.

```
pip install tqdm

from tqdm import tqdm

for url in tqdm(list_of_urls):
    response = session.get(url)
```

76% | ██████████ | 7568/10000 [00:33<00:10, 229.00it/s]

Часть строки	Значение
76%	общий процент выполнения
7568 / 10000	пройдено итераций / всего итераций
00:33<00:10	время выполнения / расчётное время до конца
229.00 it/s	количество итераций в секунду



Собранные данные удобно сразу сохранять в **csv**-файл — стандартный табличный формат:

```
import csv

lst = ['one', 'two', 'three']

with open('res.csv', 'w', newline='', encoding='utf-8-sig') as file:
    writer = csv.writer(file, delimiter=';')
    writer.writerow(lst)
```

Параметр	Зачем нужен
<code>newline=''</code>	указывать всегда — иначе строки могут интерпретироваться неверно и документ «сползёт»
<code>encoding='utf-8-sig'</code>	open() по умолчанию использует unicode ; для нужной кодировки указываем явно
<code>delimiter</code>	разделитель между элементами строки — можно указать любой



```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, 'lxml')

soup.find(name, attrs)      # первый элемент
soup.find_all(name, attrs)  # список элементов
soup.select(css)            # по CSS-селектору

tag.text / tag.get_text()   # текст
tag['attr'] / tag.get('attr') # атрибут
```

Перед разбором страницы:

- выбирайте **lxml** как парсер для скорости;
- ищите теги по **id** и надёжным атрибутам;
- двигайтесь от крупных блоков к мелким;
- используйте **requests-cache** при отладке;
- добавьте **tqdm**, если страниц много;
- сохраняйте результат в **csv** сразу по ходу сбора.

Итог: Requests получает **HTML**-код страницы, а **BeautifulSoup** превращает его в дерево тегов, по которому легко находить и извлекать нужные данные.